

Charity Donor Outreach Skill Case Study

Natarajan Lekshmi Narayana Pillai

The draft skill has three classes of problems. It instructs the agent to deceive donors, most seriously with a matching gift promise that may not exist. It cannot produce consistent output, because every calculation is left to the language model and the rules are ambiguous about rounding order, recency, and how modifiers stack. And it cannot scale, because the donor data is embedded inside the very instructions that also tell the agent to read an uploaded file. My rewrite removes every unverifiable claim, moves all math into a deterministic script covered by tests, and makes the uploaded CSV the single source of truth, with a review summary and draft banners keeping a human between the system and the donor.

The first section below walks through each improvement, why I made it, and its impact. The second section describes the rewritten skill, delivered as a zip archive with SKILL.md as the entry point. I treated the mocked donor data as given.

Part 1. Improvements and their impact

1. **Remove the deceptive content.** The emergency template tells donors their gift will be matched even when no match is confirmed. That is deceptive solicitation. It exposes the charity to legal and reputational risk, and it collapses donor trust the first time someone asks about the match. Smaller versions of the same problem appear elsewhere. Guessing titles from first names risks misgendering donors. Assigning a relationship manager invites inventing a staff member. The event template cites a registration count nobody confirmed. In the rewrite, match language and headcounts appear only when staff pass in confirmed details, titles are used only when present in the data, and the signer must be a real person supplied as configuration. The impact is that no letter can make a claim the organization cannot stand behind.
2. **Separate donor data from instructions.** The skill embeds a 50 row donor table while also telling the agent to read an uploaded CSV, so there are two conflicting sources of truth inside the same workflow. Embedded data goes stale, cannot scale past a demo, bloats the context on every run, and hardcodes donor PII into a prompt file. I made the uploaded CSV the only data source and moved the mock table into examples/sample_donors.csv as a test fixture. The impact is a skill that scales with the donor list and keeps personal data out of the instructions.
3. **Move the math into a deterministic script.** Asking a language model to run a five step formula across hundreds of donors invites silent arithmetic errors and drift between runs, which defeats the stated goals of consistency and reliability. I moved segmentation, ask calculation, and letter assembly into scripts/generate_letters.py, and every rule is covered by scripts/run_tests.py. Identical inputs now always produce identical letters, and the logic is testable and auditable. The agent's job shrinks to gathering inputs, running the script, and reporting what was flagged. The model still owns the judgment work. An optional enrichment step lets it adapt each campaign

paragraph for tone using only facts already in that donor's row, so the writing stays flexible while the numbers stay deterministic.

4. **Resolve the ambiguities in the ask formula.** The original rounds at step 3 and then keeps multiplying, so final asks are not round numbers. The phrase "gave last year" is undefined without a reference date. The rules never say whether Bronze or Lapsed donors receive the volunteer bonus or the emergency multiplier, or how halves round. The rewrite rounds last, defines recency against an explicit reference date that defaults to today, and documents every decision in one ordered list. The impact is that two runs or two staff members can no longer interpret the rules differently.
5. **Fix the tier model.** Lapsed is treated as a tier when it is really a recency status, and the sample data contradicts its own tier definitions in four rows. Ruth Andersen has \$25,000 in lifetime giving yet sits in Silver. The bands also leave gaps as written, since Platinum starts over \$50,000 while Gold stops at \$49,999, so a donor at exactly \$50,000 belongs to no tier. The rewrite computes tiers from lifetime giving with inclusive lower bounds so every amount lands in exactly one tier, keeps a CSV supplied tier when present because staff may have CRM context the file does not show, and flags disagreements as tier_mismatch. Lapsed status is derived from the reference date. Lapsed Gold and Platinum donors are pulled out of the bulk run entirely and flagged high_value_lapsed, because a \$50 form letter to a donor with \$145,000 in lifetime giving damages the relationship. Those donors are routed to personal outreach by staff. The impact is segmentation that matches the data instead of contradicting it, plus protection for the most valuable relationships.
6. **Replace "make reasonable assumptions" with validation and human review.** Guessing missing fields is how a wrong name or a wrong amount reaches a real donor. The rewrite skips incomplete rows with a stated reason, writes a review_summary.csv with every computed value and flag, stamps a visible draft banner into each letter, and requires the agent to surface every flag in its report. I also added an ask_below_last_gift guardrail. The percentage formula produces asks below recent giving for most active donors, so I implemented the formula exactly as specified but flagged those rows to give the fundraising team evidence to revisit the percentages. The impact is that nothing questionable slips through silently and every letter passes a human before it is sent.
7. **Scope the trigger description.** The current description fires on money, emails, reports, or any communication task, so the skill would hijack unrelated work across a workspace. The rewrite states exactly when to use it, generating donor solicitation letters from a donor list, and exactly when not to, such as general email drafting, grant writing, and internal reports. The impact is a skill that activates when it should and stays out of the way when it should not.
8. **Write files instead of chat output, and use the whole schema.** Hundreds of letters dumped into a chat window are unusable and unreviewable. The script writes one HTML file per donor plus the review summary, ready for mail merge. The rewrite also puts the previously unused region field to work in the campaign paragraph, computes a consecutive year giving streak for annual fund letters, and sources every template placeholder from the data or the confirmed configuration so nothing

like [DONATION_URL] can leak into a real letter. The impact is output that staff can actually review, edit, and send at scale.

Part 2. The rewritten skill

The rewrite follows standard skill anatomy. SKILL.md carries the scoped description, the workflow, the guardrails, and a readable summary of the business rules. scripts/generate_letters.py is the single source of truth for segmentation and ask math. assets/letter_template.html is the letter template. examples/sample_donors.csv holds the original mocked donor table as a runnable test fixture, and scripts/run_tests.py covers every rule with unit checks plus an end to end run that reproduces the exact numbers below. The skill declares that it requires Python execution, and its guardrails forbid falling back to hand computed math if scripts are unavailable, since that would quietly reintroduce the problem the design removes. A typical run looks like this.

```
python scripts/generate_letters.py \  
  --csv donors.csv --campaign annual \  
  --donation-url "https://example.org/donate" \  
  --signer-name "Jane Doe" --signer-title "Development Director" \  
  --as-of 2024-12-31 --outdir output
```

I verified the rewrite end to end against the sample data with an annual fund run, using a reference date of December 31, 2024 to match the vintage of the mocked data. From 50 rows the script produced 48 draft letters, routed Robert Svensson and Walter Adeyemi to personal outreach as high value lapsed donors, caught all four tier mismatches present in the sample data, and flagged 30 rows where the computed ask falls below the donor's most recent gift. Spot checks matched hand calculations. Earl Fontaine's ask computes to \$36,100. A second run confirmed the emergency multiplier and that match language appears only when confirmed details are supplied. All of these checks now live in scripts/run_tests.py, so anyone can reproduce them with one command.

Every letter is generated as a draft with a visible review banner, and the review summary gives staff one place to check every number and flag before anything reaches a donor.

If this were headed to production, the design already sets up the next steps. The fixture and its golden numbers guard against regressions as the rules evolve. A pilot batch reviewed by the fundraising team validates tone before any real send. And tracking response rates per segment would provide the data to tune the tier percentages that the ask_below_last_gift flags are already calling into question.